



ZAMBIA UNIVERSITY COLLEGE OF TECHNOLOGY

School of Information and Communication Technology

Department of Information Technology

PASSWORD MANAGER WITH ZERO-KNOWLEDGE ENCRYPTION

*A Project Proposal submitted to Zambia University College of Technology as a partial fulfilment for
the award of a Bachelor of Information Technology*

By

NANKOLOWE RITAH 2410802

KABASO KELVIN 2410780

CHANDA CHANDA 2410902

SUPERVISOR: _____

SIGNATURE: _____

NDOLA, ZAMBIA 2026

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION	3
1.1 Introduction and Project Background	3
1.2 Problem Statement	4
1.3 Research Objectives	5
1.4 Research Questions	5
1.5 Scope of the Study	6
1.6 Significance of the Study	6
1.7 Conceptual Framework	7
1.8 Summary	8
CHAPTER 2: LITERATURE REVIEW	10
2.1 Introduction	10
2.2 Related Works	10
2.3 Review of Related Works	11
2.4 Proposed System	12
2.5 Summary	12
CHAPTER 3: METHODOLOGY	14
3.1 Introduction	14
3.2 Software Development Methodology	14
3.3 Requirements Specification	14
3.4 Software Design Specifications	15
3.5 Software Development Tools	16
3.6 Research Timeline	16
3.7 Research Budget	17
3.8 Summary	17
REFERENCES	19

CHAPTER 1: INTRODUCTION

1.1 Introduction and Project Background

Password security remains one of the most pervasive and consequential cybersecurity challenges in Zambia and globally. As individuals and organisations accumulate dozens of online accounts across banking, government, social media, and productivity platforms, the psychological burden of managing unique, strong credentials for each service becomes unmanageable. The predictable human response reusing passwords across multiple accounts or storing them in unencrypted spreadsheets, browser-saved lists, or plain-text documents creates systemic security vulnerabilities that adversaries routinely exploit through credential-stuffing attacks, phishing campaigns, and database breach dumps. A survey of cybersecurity practices among Zambian small and medium enterprises by Mbewe and Phiri (2023) found that 68% of respondents stored passwords in plain-text formats, and 71% reused the same password across three or more accounts.

Password managers offer a technically sound solution to this problem by storing all credentials in a single encrypted vault, requiring users to remember only one master password. However, the majority of commercial password managers including widely used services such as LastPass and 1Password employ server-side encryption models in which the service provider manages encryption keys on behalf of users. This architecture introduces a fundamental trust requirement: users must trust that the service provider will not access their credentials, that the provider's servers will not be breached, and that the provider will not be compelled by legal process to surrender credential data. The LastPass breach of 2022, in which encrypted vault data belonging to millions of users was exfiltrated, demonstrated the real-world consequences of this server-side trust model even when encryption is in place.

Zero-knowledge encryption provides a technically superior alternative. In a zero-knowledge model, all encryption and decryption operations are performed exclusively on the client device the user's browser or desktop application using a key derived from the master password via a key derivation function such as PBKDF2. The server receives and stores only ciphertext, never the plaintext credentials or the encryption key. Even a complete server compromise therefore exposes only opaque ciphertext that cannot be decrypted without the user's master password. This architecture is described as zero-knowledge because the service provider has zero knowledge of the user's encryption key or plaintext credentials.

This project proposes the design, development, and evaluation of a cross-platform desktop Password Manager implementing zero-knowledge AES-256 encryption and PBKDF2 key derivation, built using the Electron framework with a React.js user interface. The vault will be stored locally in an encrypted SQLite database. The system will be evaluated through security testing including static analysis, memory inspection, and brute-force resistance benchmarking, and through usability testing with a sample of Zambian university students and staff.

1.2 Problem Statement

Password security in Zambia presents four interconnected technical and behavioural challenges that existing tools have not adequately addressed for the local context. First, the widespread practice of storing passwords in plain-text formats spreadsheets, text files, and browser-saved lists leaves credentials exposed to any party with physical or digital access to the device, with no encryption barrier between the stored data and an attacker. Second, most commercial password managers employ server-side encryption with provider-managed keys, meaning that a service provider breach, legal warrant, or insider threat can expose user credentials regardless of whether the service itself has been

properly secured. Third, strong key derivation the process of computationally stretching a master password into a cryptographically strong encryption key using thousands of PBKDF2 iterations is absent from most locally understood credential storage approaches, leaving vault data vulnerable to brute-force key recovery attacks if the encrypted database file is obtained. Fourth, no locally developed, auditable, zero-knowledge credential storage tool exists for Zambian individual or organisational users, leaving this population entirely dependent on foreign commercial services whose security practices and legal jurisdictions may not align with Zambian data protection interests.

1.3 Research Objectives

The study aims to achieve the following three objectives:

1. To implement a PBKDF2-based key derivation module that generates a cryptographically strong session encryption key from the user's master password with configurable iteration counts to resist brute-force key recovery attacks.
2. To design and implement a client-side AES-256 encryption engine that encrypts all vault entries exclusively on the user's device before any storage, ensuring the application server never receives or stores plaintext credentials.
3. To build and evaluate a cross-platform desktop password vault using Electron and React.js with encrypted SQLite storage, validated through security testing and usability evaluation with a sample of Zambian users.

1.4 Research Questions

The study is guided by the following research questions:

4. How can a PBKDF2 key derivation module be implemented to generate session-specific encryption keys from a master password with iteration counts sufficient to resist contemporary brute-force attacks?
5. How can a client-side AES-256 encryption engine be designed and implemented to ensure that all vault entries are encrypted on the user's device before storage, with the server retaining zero knowledge of plaintext data?
6. How does the Electron and React.js password vault perform in terms of AES-256 encryption correctness, PBKDF2 key derivation resistance, and usability as evaluated through security testing and user feedback?

1.5 Scope of the Study

This study covers the design, development, and security and usability evaluation of a zero-knowledge password manager desktop application. The system scope includes master password authentication with PBKDF2 key derivation, AES-256 client-side encryption of all vault entries, local encrypted SQLite vault storage, a React.js-based Electron desktop interface for adding, viewing, editing, and deleting credentials, a built-in password generator with configurable length and complexity rules, vault lock on inactivity timeout, and encrypted vault import and export functionality. The study is scoped to individual user deployment on a single device and does not cover cloud synchronisation across devices, mobile native application development, team or organisational vault sharing, biometric authentication integration, or browser extension auto-fill functionality. Security evaluation is conducted through static analysis, memory inspection during runtime, and PBKDF2 brute-force resistance benchmarking. Usability evaluation is conducted through structured testing with a sample of 15 university students and staff.

1.6 Significance of the Study

Zero Server Trust Architecture: The zero-knowledge design guarantees that even a complete server-side compromise exposes only AES-256 ciphertext computationally infeasible to decrypt without the master password. This represents a categorical security improvement over server-managed encryption models.

Strong Key Security via PBKDF2: PBKDF2 with a high iteration count makes deriving the encryption key from a stolen encrypted vault file computationally expensive for an attacker, providing meaningful resistance to offline brute-force key recovery attacks.

User Data Sovereignty: By storing credentials exclusively in a locally encrypted database on the user's own device, the system returns full control of credential data to the user eliminating dependence on foreign service providers whose security practices and legal obligations may be opaque.

Addressing the Zambian Credential Security Gap: The study directly responds to the documented prevalence of plain-text password storage in Zambia, providing an auditable, open-source alternative that requires no subscription or trust in a foreign entity.

Practical Cryptography Demonstration: The project bridges the gap between theoretical cryptography education and working implementation, demonstrating how AES-256 and PBKDF2 function in a real consumer product contributing applied cryptography knowledge to Zambian computing curricula.

Academic Contribution: The study produces a documented zero-knowledge architecture implementation in Electron and React.js with a reproducible security evaluation methodology, contributing to the applied cybersecurity research literature from a Zambian academic institution.

1.7 Conceptual Framework

The proposed system is structured as a four-stage pipeline that transforms a user's master password and plaintext credentials into securely encrypted vault storage. Figure 1.1 illustrates this framework.

The User actor interacts with the Electron/React UI the desktop application interface through which users authenticate with their master password, add and retrieve credentials, and configure vault settings. The UI communicates bidirectionally with the Vault Manager, the core application logic layer responsible for coordinating all encryption operations, vault read/write transactions, and session state management. The Vault Manager communicates bidirectionally with the Encrypted SQLite Store the local database file that stores all vault data exclusively as AES-256 ciphertext, never in plaintext.

Below the Vault Manager, the AES-256 Encryption module handles all encryption and decryption operations using the session key, and communicates with the PBKDF2 Key Derivation module, which derives the AES-256 session key from the user's master password using a stored random salt and a configured number of iterations. The derivation chain ensures that the session key is never stored it exists only in memory during an active session and is discarded on vault lock or application close.

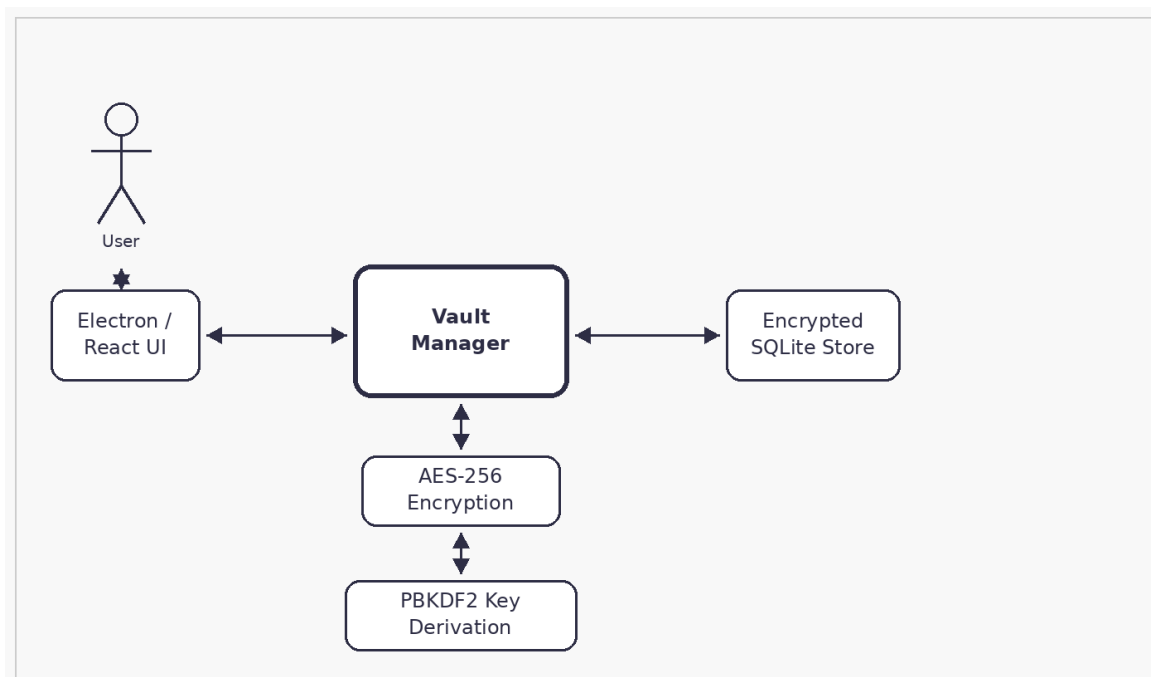


Figure 1.1: Conceptual Framework Password Manager with Zero-Knowledge Encryption

1.8 Summary

This chapter established the rationale for a zero-knowledge password manager in the Zambian context, articulated a four-part problem statement covering plain-text storage, server-side key management, absent key derivation, and the local tool gap, defined three measurable objectives and research questions, scoped the system to individual desktop deployment with AES-256 and PBKDF2, and described the four-stage conceptual framework from master password input through key derivation and encryption to encrypted local storage.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

This chapter reviews literature relevant to the proposed zero-knowledge password manager across three themes: the evolution of password security and credential management practices, asymmetric and symmetric cryptographic approaches to credential storage, and password manager vulnerability research. The review identifies the gaps that the proposed system addresses.

2.2 Related Works

2.2.1 Password Security and Credential Management

Bonneau, Herley, van Oorschot, and Stajano (2012) conducted a comprehensive evaluation of password alternatives in their landmark paper *The Quest to Replace Passwords*, systematically assessing 35 proposed authentication schemes against a framework of security, usability, and deployability criteria. Their analysis concluded that despite well-documented security weaknesses, passwords combined with a secure credential manager represented the most deployable practical solution for the foreseeable future establishing the enduring relevance of password manager research.

Their framework, however, was evaluated in a theoretical context without addressing the specific zero-knowledge architecture requirements needed to protect credentials from provider-side compromise a gap the current project directly addresses through client-side encryption with PBKDF2 key derivation.

2.2.2 Cryptographic Approaches to Secure Credential Storage

Kelsey, Schneier, Hall, and Ferguson (1998) analysed secure hash constructions and key derivation functions, establishing the theoretical basis for PBKDF-style approaches that computationally stretch low-entropy passwords into high-entropy cryptographic keys. Their work provided the mathematical

foundation for PBKDF2, later standardised in NIST SP 800-132 (2010), which prescribes minimum iteration counts and salt requirements for password-based key derivation in sensitive applications.

NIST SP 800-38D (2007), specifying AES-GCM (Galois/Counter Mode) authenticated encryption, established the standard for combining AES-256 confidentiality with integrity protection ensuring that any tampering with vault ciphertext is detectable on decryption. The current project implements AES-256-GCM as the authenticated encryption scheme for vault entries, directly applying this standard.

2.2.3 Password Manager Vulnerability Research

Li, He, Xu, and Chen (2020) conducted a systematic security analysis of five major commercial password managers, identifying vulnerabilities including clipboard-based credential exposure, insecure memory handling (credentials persisting in process memory after vault lock), and inadequate master password verification that allowed offline brute-force attacks against exported vault files. Their findings provide a direct checklist of vulnerabilities the current project must address and avoid in its implementation.

Mbewe and Phiri (2023) surveyed cybersecurity practices among Zambian SMEs, finding that 68% stored passwords in plain-text formats and that awareness of password manager security was low. Their survey establishes the local demand context for the proposed tool and provides baseline data against which the usability evaluation of the current project can be contextualised.

2.3 Review of Related Works

The reviewed works establish three findings directly relevant to the current project. First, password managers remain the most deployable practical credential security solution, but existing commercial

managers introduce server-side trust requirements that the zero-knowledge model of the current project eliminates. Second, PBKDF2 and AES-256-GCM represent the standard-endorsed approaches for password-based key derivation and symmetric authenticated encryption respectively both adopted by the proposed system. Third, the specific vulnerabilities identified by Li et al. (2020) clipboard exposure, memory persistence, and offline brute-force susceptibility provide a concrete security evaluation checklist for the current project, and the prevalence of plain-text storage in Zambia documented by Mbewe and Phiri (2023) establishes the practical urgency of the proposed solution.

2.4 Proposed System

The proposed system addresses the identified gaps through four specific design decisions: client-side-only encryption using AES-256-GCM ensuring the server has zero knowledge of plaintext data, PBKDF2-SHA256 key derivation with a minimum of 600,000 iterations as recommended by OWASP (2023) to resist offline brute-force attacks, clipboard auto-clear after 30 seconds to address the clipboard exposure vulnerability identified by Li et al., and session key purging from memory on vault lock or application close to address the memory persistence vulnerability. All design decisions are directly traceable to findings in the reviewed literature.

2.5 Summary

This chapter reviewed password security literature, NIST cryptographic standards for key derivation and authenticated encryption, and password manager vulnerability research, establishing the technical foundation for all design decisions in the proposed system. The review identified specific security vulnerabilities the project must address, confirmed the appropriateness of the AES-256-GCM and

PBKDF2-SHA256 approach, and contextualised the project within the Zambian credential security landscape.

CHAPTER 3: METHODOLOGY

3.1 Introduction

This chapter outlines the methodology for designing, developing, and evaluating the zero-knowledge password manager. It covers the development model, requirements, design tool, development stack, timeline, and budget.

3.2 Software Development Methodology

The project adopts Agile with security-first sprints. Each sprint begins with a STRIDE threat modelling session to identify threats specific to the sprint's target module before implementation begins. OWASP Secure Coding Practices are applied throughout. Four sprints are planned: Sprint 1 covers PBKDF2 key derivation module and AES-256-GCM encryption engine. Sprint 2 covers the SQLite vault schema, read/write operations, and session management. Sprint 3 covers the Electron/React.js desktop interface, password generator, and import/export. Sprint 4 covers security testing, usability evaluation, and documentation.

3.3 Requirements Specification

3.3.1 Functional Requirements

The following functional requirements were identified:

7. The system shall authenticate users via a master password and derive a session AES-256 key using PBKDF2-SHA256 with a minimum of 600,000 iterations and a random per-vault salt.
8. The system shall encrypt all vault entries using AES-256-GCM on the client device before writing to the SQLite database file.
9. The system shall support add, view, edit, and delete operations for vault entries including title, username, password, URL, and notes fields.
10. The system shall generate random passwords with configurable length (8–64 characters) and character set options.

11. The system shall copy passwords to the operating system clipboard and automatically clear the clipboard after 30 seconds.
12. The system shall lock the vault and purge the session key from memory after a configurable inactivity timeout.
13. The system shall export the vault as an encrypted file and import vault data from a compatible encrypted export file.

3.3.2 Non-Functional Requirements

The following non-functional requirements were identified:

14. Security: No plaintext credentials or encryption keys shall be written to disk or transmitted over any network interface under any operating condition.
15. Performance: Vault unlock (PBKDF2 derivation) shall complete within 3 seconds on a standard consumer laptop. Individual vault entry encryption/decryption shall complete within 200 milliseconds.
16. Compatibility: The application shall run on Windows 10/11 and macOS 12+ via Electron.
17. Usability: All core operations (unlock, add entry, retrieve password) shall be completable in under 5 user interactions.

3.4 Software Design Specifications

3.4.1 Design Tool

The primary design tool is the UML Use Case Diagram. The Use Case Diagram was selected to clearly define the boundary between the User actor and the system's core functions master password authentication, vault CRUD operations, password generation, and import/export and to specify which operations require an authenticated session, establishing the security boundary for requirements validation.

3.4.2 Design Diagram

A UML Use Case Diagram will be produced showing the User actor and their interactions with the following use cases: Unlock Vault, Add Credential, View Credential, Edit Credential, Delete Credential, Generate Password, Copy to Clipboard, Lock Vault, Export Vault, and Import Vault. The

diagram will show the include relationship between Unlock Vault and Derive Key (PBKDF2) and between Add/Edit Credential and Encrypt Entry (AES-256).

3.5 Software Development Tools

- Electron (v28+): Cross-platform desktop application framework enabling the React.js UI to run as a native desktop application on Windows and macOS.
- React.js: Component-based UI framework for building the vault interface, login screen, and settings panels.
- Node.js Web Crypto API: Browser-standard cryptographic API used for AES-256-GCM encryption/decryption and PBKDF2-SHA256 key derivation, avoiding third-party cryptographic dependencies.
- better-sqlite3: Synchronous SQLite binding for Node.js used for encrypted local vault storage.
- SQLCipher: SQLite extension providing transparent AES-256 encryption of the entire database file at rest.
- Jest: JavaScript testing framework for unit testing key derivation, encryption/decryption correctness, and vault CRUD operations.
- Git and GitHub: Version control and remote repository for sprint-level development tracking.

3.6 Research Timeline

Table 3.1 presents the project timeline spanning April to November 2026.

Activity	Phase	Start Week	End Week	Duration
Proposal Writing	Planning	Week 1	Week 4	4 weeks
Literature Review	Planning	Week 1	Week 8	8 weeks
Requirements Gathering	Analysis	Week 5	Week 10	6 weeks
System Design (UML)	Design	Week 9	Week 14	6 weeks
Crypto Module Development	Build	Week 13	Week 22	10 weeks
Electron App Development	Build	Week 17	Week 26	10 weeks

Activity	Phase	Start Week	End Week	Duration
Security & Usability Testing	Testing	Week 24	Week 29	6 weeks
Documentation & Report	Delivery	Week 28	Week 32	5 weeks

Table 3.1: Research Timeline Password Manager with Zero-Knowledge Encryption

3.7 Research Budget

Table 3.2 presents the estimated research budget.

#	Item / Description	Unit	Estimated Cost (ZMK)
1	VPS Hosting sync server + auth endpoint (6 months)	6 mo	K 1,260
2	Domain + SSL Certificate	1	K 780
3	Usability Testing Incentives (15 participants × K60)	15	K 900
4	Transportation testing sessions and interviews	Lump	K 1,500
5	Research Materials IEEE Xplore, NIST publications	1	K 1,200
6	Printing proposal, interim and final reports	9	K 480
7	Contingency	1	K 880

TOTAL: K 7,000

Table 3.2: Research Budget Password Manager with Zero-Knowledge Encryption

3.8 Summary

This chapter described the Agile security-first methodology with STRIDE threat modelling, specified functional and non-functional requirements, selected the UML Use Case Diagram as the design tool,

documented the Electron/React.js/Web Crypto API stack, and presented a weekly timeline and budget of K 7,000.

REFERENCES

- [1]Bonneau, J., Herley, C., van Oorschot, P.C. & Stajano, F. (2012). The Quest to Replace Passwords: A Framework for Comparative Evaluation of Web Authentication Schemes. IEEE Symposium on Security and Privacy.
- [2]Kelsey, J., Schneier, B., Hall, C. & Ferguson, N. (1998). Secure Applications of Low-Entropy Keys. Information Security Workshop, Lecture Notes in Computer Science.
- [3]NIST SP 800-132. (2010). Recommendation for Password-Based Key Derivation Part 1: Storage Applications. National Institute of Standards and Technology.
- [4]NIST SP 800-38D. (2007). Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM). National Institute of Standards and Technology.
- [5]Li, Z., He, W., Akhawe, D. & Song, D. (2020). The Emperor's New Password Manager: Security Analysis of Web-based Password Managers. USENIX Security Symposium.
- [6]Mbewe, T. & Phiri, C. (2023). Cybersecurity Awareness and Password Management Practices in Zambian SMEs. Zambia ICT Security Journal.
- [7]OWASP. (2023). Password Storage Cheat Sheet. <https://cheatsheetseries.owasp.org>